Rigid-Body Transformations

Where a number is measured from, and how to move it somewhere else

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 7, Lecture A

What this lecture does

Roadmap

A robot measures everything from somewhere. Today: how to say *where from*, how to move a measurement between two places, and why chaining these correctly is the single most reusable piece of mathematics in robotics.

1. Why frames exist
2. Rotation
3. Rotation and translation together
4. Trees, and time

Outline

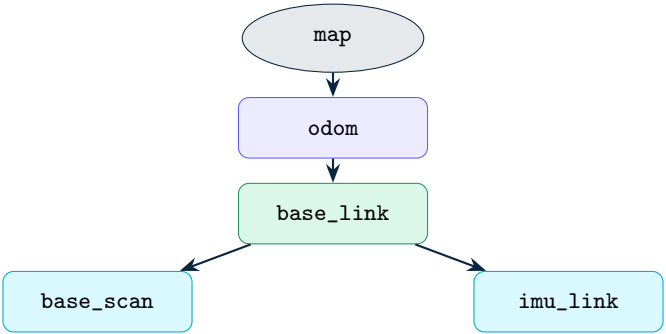
A number with no frame is not information

The LiDAR reports an obstacle at **2.1 m, bearing** 30°.

2.1 m from what?	The laser, which sits 10 cm forward and 20 cm up from the robot's centre
30° from what?	The laser's zero direction, which may not be the robot's forward
So where is it?	**Unanswerable** until you say what the numbers are measured from

A **frame** is that "measured from". Every position, velocity and orientation in robotics belongs to exactly one, and a value quoted without its frame is not wrong — it is meaningless.

The frames on a small mobile robot



`map`	The world. Fixed, never drifts. Jumps when localization corrects itself.
`odom`	Where the robot thinks it has driven. Smooth, but drifts forever.
`base_link`	The robot's own centre. Everything on the robot hangs off this.
`base_scan`	The laser. Bolted on, so this link never changes.

The two questions this lecture answers

1. **Expression.** I have a point in `base_scan`. What is it in `base_link`?
2. **Composition.** I know `base_scan` relative to `base_link`, and `base_link` relative to `odom`. What is `base_scan` relative to `odom`?

Both have one answer, and it is a matrix. Once you can write it down and multiply it in the right order, obstacle avoidance, mapping, localization and navigation all become bookkeeping.

In the field

Almost every "the robot turned the wrong way" bug in this course will turn out to be a frame error — a transform applied backwards, or applied in the wrong order. Not a control bug. A frame bug.

Outline

Rotating a point in the plane

Rotate $\mathrm{p} = (x, y)$ by angle θ about the origin:

$$\mathrm{p}' \;=\; R(\theta)\,\mathrm{p}, \qquad R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Read the columns: they are where the old axes end up.

$$R(\theta)\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}\cos\theta\\ \sin\theta\end{bmatrix}, \qquad R(\theta)\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}-\sin\theta\\ \cos\theta\end{bmatrix}$$

A check you can do in your head

Put $\theta = 90^\circ$. Then $\hat{x} = (1, 0)$ maps to $(0, 1)$ — forward becomes left, which is what turning left should do. If a rotation matrix ever surprises you, test it with 90° and one axis.

Three properties that matter

$R^{\mathsf{T}} R = I$	The columns are unit length and perpendicular. Rotation does not stretch or skew.
$R^{-1} = R^{\mathsf{T}}$	**The inverse is free.** Undoing a rotation is a transpose, not a matrix inversion.
$\det R = +1$	It is a rotation, not a reflection. A -1 here means you have accidentally mirrored your robot.

In the field

$R^{-1} = R^{\mathsf{T}}$ is not a curiosity — it is why a robot can invert transforms thousands of times a second on a small processor. Every lookup in the opposite direction uses it.

The confusion everyone has once

Rotating the *point*

The frame stays. The object moves.

Turn the obstacle 30° about the robot.

$\mathrm{p}' = R(\theta)\,\mathrm{p}$

Re-expressing in a rotated *frame*

The object stays. The observer turns.

The obstacle did not move; the laser is mounted 30° off.

$\mathrm{p}' = R(\theta)^{\mathsf{T}}\,\mathrm{p}$

Same matrix. Opposite direction. These differ by a transpose, and mixing them up produces a robot that turns away from the goal instead of towards it.

In this course we almost always want the second one: nothing moved, we are changing where we are looking from.

Outline

The obvious formula, and why we abandon it

A rigid transform is a rotation then a translation:

$$\mathrm{p}_A \;=\; R\,\mathrm{p}_B + \mathrm{t}$$

Correct. Now chain three of them:

$$\mathrm{p}_A \;=\; R_1(R_2(R_3\mathrm{p}_D + \mathrm{t}_3) + \mathrm{t}_2) + \mathrm{t}_1$$

Composing two of them	Not a multiplication. A rule you must remember.
Inverting one	$\mathrm{p}_B = R^{\mathsf{T}}(\mathrm{p}_A - \mathrm{t})$ — easy to get wrong
Chaining four	Unreadable, and unmaintainable

We want one object that composes by multiplication. That is the next slide.

Homogeneous coordinates

Add a 1 to the point, and a row to the matrix:

$$\begin{bmatrix} \mathrm{p}_A \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R & \mathrm{t} \\ 0^{\mathsf{T}} & 1 \end{bmatrix}}_{T} \begin{bmatrix} \mathrm{p}_B \\ 1 \end{bmatrix}$$

In the plane that is 3×3; in 3D, 4×4. Rotation and translation now live in one object.

Compose	$T_{AC} = T_{AB}\, T_{BC}$	(matrix multiplication)
Invert	$T^{-1} = \begin{bmatrix} R^{\mathsf{T}} & -R^{\mathsf{T}}\mathrm{t} \\ 0^{\mathsf{T}} & 1 \end{bmatrix}$	

The extra 1 is what lets a translation — which is not linear — be written as a matrix. That is the entire trick.

Notation discipline, or you will lose an afternoon

$$T_{AB} \;=\; \text{the pose of frame } B \text{ expressed in frame } A$$
$$=\; \text{the thing that turns } B\text{-coordinates into } A\text{-coordinates}$$

Written this way, composition reads itself — **the inner indices cancel**:

$$T_{AC} \;=\; T_{AB}\,T_{BC} \qquad\qquad T_{BA} \;=\; T_{AB}^{-1}$$

Use this as a type check

If the inner subscripts do not match, the product is meaningless — no matter how well the dimensions line up. $T_{AB}\,T_{CD}$ compiles, runs, and is nonsense. This notation is the only defence, and it costs nothing to adopt.

Worked: where is that obstacle, really?

The laser sits 10 cm forward and 20 cm up from the robot centre, mounted straight. It reports a point at $(2.1, 0)$ in `base_scan`.

$$T_{\text{base\_link},\,\text{base\_scan}} = \begin{bmatrix} 1 & 0 & 0.10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{p}_{\text{scan}} = \begin{bmatrix} 2.1 \\ 0 \\ 1 \end{bmatrix}$$

$$\mathbf{p}_{\text{base}} = T_{\text{base\_link},\,\text{base\_scan}}\,\mathbf{p}_{\text{scan}} = \begin{bmatrix} 2.20 \\ 0 \\ 1 \end{bmatrix}$$

Ten centimetres. It sounds negligible — until Week 11, where the robot decides whether it fits through a gap. **Ten centimetres is the difference between passing and scraping.**

Outline

Why a tree, and not just a pile of transforms

Every frame has **exactly one parent**. That single rule buys three things:

One path	Between any two frames there is exactly one chain, so the answer is unambiguous
Automatic lookup	The library walks the chain and multiplies; you never write the product
Errors are findable	Two publishers of the same parent–child link is detectable, and is always a bug

In the field

"Two nodes publishing `odom` → `base_link`" is one of the most common faults on a real robot: the transform flickers between two answers and the robot twitches. Because it is a tree, the tooling can tell you — which it could not if any frame could have several parents.

Three dimensions: the same idea, one hard part

In 3D the transform is 4×4 and everything above still holds. The only genuinely new difficulty is **how to write the rotation**:

Form	Numbers	The catch
Matrix	9	Correct, composes cleanly, but 9 numbers for 3 degrees of freedom
Euler angles	3	Readable — and suffers *gimbal lock*: at certain orientations two axes collapse and you lose a degree of freedom
Quaternion	4	No gimbal lock, cheap to compose and interpolate; not human-readable

ROS 2 stores orientations as quaternions, everywhere, for exactly this reason.

You will convert yaw $\leftrightarrow$ quaternion constantly and read the quaternion itself almost never. That is normal, and it is not a gap in your understanding.

A transform is only true at an instant

`base_link` → `base_scan` is bolted metal: constant forever.

`odom` → `base_link` changes every time the wheels turn. Asking "where was the robot?" without saying *when* has no answer.

Why Week 2's `Header` exists

The library stores a short history and answers "what was this transform at time t?", by interpolating between the samples on either side. That is only possible because every message carries the time its data was *measured*. This is Week 1's "nothing is instantaneous" made concrete.

Two failures that come from ignoring time

What you asked for	What happens
A transform at a time *later* than anything received	"Lookup would require extrapolation into the future." The library refuses to guess — correctly.
A transform at a time already discarded from the buffer	"Lookup would require extrapolation into the past." Your node was too slow, or the buffer is too short.

These two messages will be among the most familiar strings of your semester. **They are not bugs in the library.** They are the library declining to invent data.

You will cause both on purpose in session B.

What to carry out of this lecture

1. Every measurement belongs to a frame. A number without one is meaningless.
2. A rigid transform is one matrix: rotation and translation together.
3. $T_{AC} = T_{AB}\,T_{BC}$ — **inner indices cancel**. Use it as a type check.
4. $T_{BA} = T_{AB}^{-1}$, and for the rotation part the inverse is just a transpose.
5. Frames form a tree: one parent each, one path between any two.
6. A transform is only valid at an instant, which is why every message is stamped.

Session B: none of this by hand. TF2 does the algebra — your job is to publish the right links and ask the right questions.

Quiz 3 today.